

C. Partitioning the Array

time limit per test: 3 seconds
memory limit per test: 256 megabytes

Allen has an array $a_1, a_2, \dots, a_n$. For every positive integer k that is a divisor of n , Allen does the following:

- He partitions the array into $\frac{n}{k}$ disjoint subarrays of length k . In other words, he partitions the array into the following subarrays:

$$[a_1, a_2, \dots, a_k], [a_{k+1}, a_{k+2}, \dots, a_{2k}], \dots, [a_{n-k+1}, a_{n-k+2}, \dots, a_n]$$

- Allen earns one point if there exists some positive integer m ($m \geq 2$) such that if he replaces every element in the array with its remainder when divided by m , then all subarrays will be identical.

Help Allen find the number of points he will earn.

Input

Each test consists of multiple test cases. The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of the array a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the elements of the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the number of points Allen will earn.

Example

input	output
8	2
4	1
1 2 1 4	2
3	4
1 2 3	4
5	1
1 1 1 1 1	2
6	1
1 3 1 1 3 1	
6	
6 2 6 2 2 2	
6	
2 6 3 6 6 6	
10	
1 7 5 1 4 3 1 3 1 4	
1	
1	

Note

In the first test case, $k = 2$ earns a point since Allen can pick $m = 2$ and both subarrays will be equal to $[1, 0]$. $k = 4$ also earns a point, since no matter what m Allen chooses, there will be only one subarray and thus all subarrays are equal.

In the second test case, Allen earns 1 point for $k = 3$, where his choice for m does not matter.

E. Good Triples

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Given a non-negative integer number n ($n \geq 0$). Let's say a triple of non-negative integers (a, b, c) is good if $a + b + c = n$, and $\text{digsum}(a) + \text{digsum}(b) + \text{digsum}(c) = \text{digsum}(n)$, where $\text{digsum}(x)$ is the sum of digits of number x .

For example, if $n = 26$, then the pair $(4, 12, 10)$ is good, because $4 + 12 + 10 = 26$, and $(4) + (1 + 2) + (1 + 0) = (2 + 6)$.

Your task is to find the number of good triples for the given number n . The order of the numbers in a triple matters. For example, the triples $(4, 12, 10)$ and $(10, 12, 4)$ are two different triples.

Input

The first line of input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. Descriptions of test cases follow.

The first and only line of the test case contains one integer n ($0 \leq n \leq 10^7$).

Output

For each test case output one integer, the number of good triples for the given integer n . Order of integers in a triple matters.

Example

input	output
12	9
11	1
0	3
1	6
2	10
3	15
4	21
5	1350
3141	166375
999	29160
2718	1522435234375
9999999	3
10000000	

Note

In the first example, the good triples are $(0, 0, 11)$, $(0, 1, 10)$, $(0, 10, 1)$, $(0, 11, 0)$, $(1, 0, 10)$, $(1, 10, 0)$, $(10, 0, 1)$, $(10, 1, 0)$, $(11, 0, 0)$.

In the second example, there is only one good triple $(0, 0, 0)$.

C. Decreasing String

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Recall that string a is lexicographically smaller than string b if a is a prefix of b (and $a \neq b$), or there exists an index i ($1 \leq i \leq \min(|a|, |b|)$) such that $a_i < b_i$, and for any index j ($1 \leq j < i$) $a_j = b_j$.

Consider a sequence of strings $s_1, s_2, \dots, s_n$, each consisting of lowercase Latin letters. String s_1 is given explicitly, and all other strings are generated according to the following rule: to obtain the string s_i , a character is removed from string s_{i-1} in such a way that string s_i is lexicographically minimal.

For example, if $s_1 = \text{dacb}$, then string $s_2 = \text{acb}$, string $s_3 = \text{ab}$, string $s_4 = \text{a}$.

After that, we obtain the string $S = s_1 + s_2 + \dots + s_n$ (S is the concatenation of all strings $s_1, s_2, \dots, s_n$).

You need to output the character in position pos of the string S (i. e. the character S_{pos}).

Input

The first line contains one integer t — the number of test cases ($1 \leq t \leq 10^4$).

Each test case consists of two lines. The first line contains the string s_1 ($1 \leq |s_1| \leq 10^6$), consisting of lowercase Latin letters. The second line contains the integer pos ($1 \leq pos \leq \frac{|s_1| \cdot (|s_1| + 1)}{2}$). You may assume that n is equal to the length of the given string ($n = |s_1|$).

Additional constraint on the input: the sum of $|s_1|$ over all test cases does not exceed 10^6 .

Output

For each test case, print the answer — the character that is at position pos in string S . **Note that the answers between different test cases are not separated by spaces or line breaks.**

Example

input	output
3	abx
cab	
6	
abcd	
9	
x	
1	

C. To Become Max

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given an array of integers a of length n .

In one operation you:

- Choose an index i such that $1 \leq i \leq n - 1$ and $a_i \leq a_{i+1}$.
- Increase a_i by 1.

Find the maximum possible value of $\max(a_1, a_2, \dots, a_n)$ that you can get after performing this operation at most k times.

Input

Each test contains multiple test cases. The first line of input contains a single integer t ($1 \leq t \leq 100$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains two integers n and k ($2 \leq n \leq 1000$, $1 \leq k \leq 10^8$) — the length of the array a and the maximum number of operations that can be performed.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^8$) — the elements of the array a .

It is guaranteed that the sum of n over all test cases does not exceed 1000.

Output

For each test case output a single integer — the maximum possible maximum of the array after performing at most k operations.

Example

input	output
6	4
3 4	7
1 3 3	179
5 6	5
1 3 4 5 1	7
4 13	6
1 1 3 179	
5 3	
4 3 2 2 2	
5 6	
6 5 4 1 5	
2 17	
3 5	

Note

In the first test case, one possible optimal sequence of operations is:

$[1, 3, 3] \rightarrow [2, 3, 3] \rightarrow [2, 4, 3] \rightarrow [3, 4, 3] \rightarrow [4, 4, 3]$.

In the second test case, one possible optimal sequence of operations is:

$[1, 3, 4, 5, 1] \rightarrow [1, 4, 4, 5, 1] \rightarrow [1, 5, 4, 5, 1] \rightarrow [1, 5, 5, 5, 1] \rightarrow [1, 5, 6, 5, 1] \rightarrow [1, 6, 6, 5, 1] \rightarrow [1, 7, 6, 5, 1]$.

E. Tracking Segments

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given an array a consisting of n zeros. You are also given a set of m not necessarily different segments. Each segment is defined by two numbers l_i and r_i ($1 \leq l_i \leq r_i \leq n$) and represents a subarray $a_{l_i}, a_{l_i+1}, \dots, a_{r_i}$ of the array a .

Let's call the segment l_i, r_i *beautiful* if the number of ones on this segment is **strictly greater** than the number of zeros. For example, if $a = [1, 0, 1, 0, 1]$, then the segment $[1, 5]$ is *beautiful* (the number of ones is 3, the number of zeros is 2), but the segment $[3, 4]$ is not *beautiful* (the number of ones is 1, the number of zeros is 1).

You also have q changes. For each change you are given the number $1 \leq x \leq n$, which means that you must assign an element a_x the value 1.

You have to find the first change after which **at least one** of m given segments becomes *beautiful*, or report that none of them is beautiful after processing all q changes.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq m \leq n \leq 10^5$) — the size of the array a and the number of segments, respectively.

Then there are m lines consisting of two numbers l_i and r_i ($1 \leq l_i \leq r_i \leq n$) — the boundaries of the segments.

The next line contains an integer q ($1 \leq q \leq n$) — the number of changes.

The following q lines each contain a single integer x ($1 \leq x \leq n$) — the index of the array element that needs to be set to 1. It is guaranteed that indexes in queries are distinct.

It is guaranteed that the sum of n for all test cases does not exceed 10^5 .

Output

For each test case, output one integer — the minimum change number after which at least one of the segments will be beautiful, or -1 if none of the segments will be beautiful.

Example

input	output
6	3
5 5	-1
1 2	3
4 5	3
1 5	3
1 3	1
2 4	
5	
5	
3	
1	
2	
4	
4 2	
1 1	
4 4	
2	
2	
3	
5 2	
1 5	
1 5	
4	
2	
1	
3	
4	
5 2	
1 5	
1 3	
5	
4	
1	
2	
3	
5	
5 5	
1 5	
1 5	
1 5	
1 5	
1 4	
3	
1	
4	
3	
3 2	
2 2	
1 3	
3	
2	
3	
1	

Note

In the first case, after first 2 changes we won't have any beautiful segments, but after the third one on a segment $[1; 5]$ there will be 3 ones and only 2 zeros, so the answer is 3.

In the second case, there won't be any beautiful segments.

E. Round Dance

time limit per test: 2 seconds
memory limit per test: 256 megabytes

n people came to the festival and decided to dance a few round dances. There are at least 2 people in the round dance and each person has exactly two neighbors. If there are 2 people in the round dance then they have the same neighbor on each side.

You decided to find out exactly how many dances there were. But each participant of the holiday remembered exactly **one** neighbor. Your task is to determine what the minimum and maximum number of round dances could be.

For example, if there were 6 people at the holiday, and the numbers of the neighbors they remembered are equal $[2, 1, 4, 3, 6, 5]$, then the **minimum** number of round dances is 1:

- $1 - 2 - 3 - 4 - 5 - 6 - 1$

and the **maximum** is 3:

- $1 - 2 - 1$
- $3 - 4 - 3$
- $5 - 6 - 5$

Input

The first line contains a positive number t ($1 \leq t \leq 10^4$) — the number of test cases. The following is a description of the test cases.

The first line of the description of each test case contains a positive number n ($2 \leq n \leq 2 \cdot 10^5$) — the number of people at the holiday.

The second line of the description of each test case contains n integers a_i ($1 \leq a_i \leq n, a_i \neq i$) — the number of the neighbor that the i th person remembered.

It is guaranteed that the test cases are correct and corresponds to at least one division of people into round dances.

It is guaranteed that the sum of n for all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output two integers — the minimum and maximum number of round dances that could be.

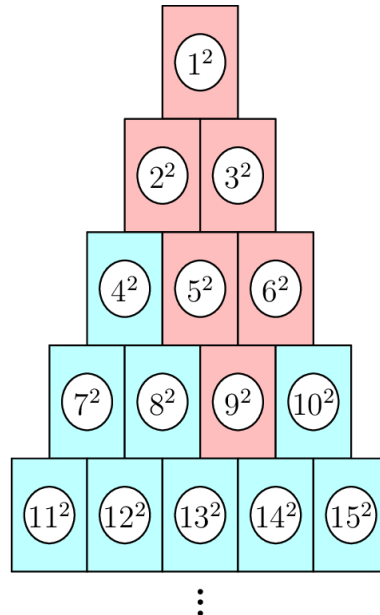
Example

input	output
10	1 3
6	2 2
2 1 4 3 6 5	1 3
6	1 1
2 3 1 5 6 4	1 2
9	1 1
2 3 2 5 6 5 8 9 8	1 1
2	2 2
2 1	1 2
4	1 1
4 3 2 1	
5	
2 3 4 5 1	
6	
5 3 4 1 1 2	
5	
3 5 4 1 2	
6	
6 3 2 5 4 3	
6	
5 1 4 3 4 2	

G. Hits Different

time limit per test: 2.5 seconds
memory limit per test: 256 megabytes

In a carnival game, there is a huge pyramid of cans with 2023 rows, numbered in a regular pattern as shown.



If can 9^2 is hit initially, then all cans colored red in the picture above would fall.

You throw a ball at the pyramid, and it hits a single can with number n^2 . This causes all cans that are stacked on top of this can to fall (that is, can n^2 falls, then the cans directly above n^2 fall, then the cans directly above those cans, and so on). For example, the picture above shows the cans that would fall if can 9^2 is hit.

What is the **sum** of the numbers on all cans that fall? Recall that $n^2 = n \times n$.

Input

The first line contains an integer t ($1 \leq t \leq 1000$) — the number of test cases.

The only line of each test case contains a single integer n ($1 \leq n \leq 10^6$) — it means that the can you hit has label n^2 .

Output

For each test case, output a single integer — the sum of the numbers on all cans that fall.

Please note, that the answer for some test cases won't fit into 32-bit integer type, so you should use at least 64-bit integer type in your programming language (like `long long` for C++). For all valid inputs, the answer will always fit into 64-bit integer type.

Example

input	output
10	156
9	1
1	5
2	10
3	21
4	39
5	46
6	146
10	63145186
1434	58116199242129511
1000000	

Note

The first test case is pictured in the statement. The sum of the numbers that fall is

$$1^2 + 2^2 + 3^2 + 5^2 + 6^2 + 9^2 = 1 + 4 + 9 + 25 + 36 + 81 = 156.$$

In the second test case, only the can labeled 1^2 falls, so the answer is $1^2 = 1$.

In the third test case, the cans labeled 1^2 and 2^2 fall, so the answer is $1^2 + 2^2 = 1 + 4 = 5$.

In the fourth test case, the cans labeled 1^2 and 3^2 fall, so the answer is $1^2 + 3^2 = 1 + 9 = 10$.

In the fifth test case, the cans labeled 1^2 , 2^2 , and 4^2 fall, so the answer is

$$1^2 + 2^2 + 4^2 = 1 + 4 + 16 = 21.$$

D. Shocking Arrangement

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$ consisting of integers such that $a_1 + a_2 + \dots + a_n = 0$.

You have to rearrange the elements of the array a so that the following condition is satisfied:

$$\max_{1 \leq l \leq r \leq n} |a_l + a_{l+1} + \dots + a_r| < \max(a_1, a_2, \dots, a_n) - \min(a_1, a_2, \dots, a_n),$$

where $|x|$ denotes the absolute value of x .

More formally, determine if there exists a permutation $p_1, p_2, \dots, p_n$ that for the array $a_{p_1}, a_{p_2}, \dots, a_{p_n}$, the condition above is satisfied, and find the corresponding array.

Recall that the array $p_1, p_2, \dots, p_n$ is called a permutation if for each integer x from 1 to n there is exactly one i from 1 to n such that $p_i = x$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 50\,000$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 300\,000$) — the length of the array a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$) — elements of the array a . It is guaranteed that the sum of the array a is zero, in other words:
 $a_1 + a_2 + \dots + a_n = 0$.

It is guaranteed that the sum of n over all test cases does not exceed 300 000.

Output

For each test case, if it is impossible to rearrange the elements of the array a in the required way, print "No" in a single line.

If possible, print "Yes" in the first line, and then in a separate line n numbers — elements $a_1, a_2, \dots, a_n$ rearranged in a valid order ($a_{p_1}, a_{p_2}, \dots, a_{p_n}$).

If there are several possible answers, you can output any of them.

Example

input	output
7	Yes
4	-5 -2 3 4
3 4 -2 -5	Yes
5	-3 2 -3 2 2
2 2 2 -3 -3	Yes
8	1 1 1 -3 1 1 1 -3
-3 -3 1 1 1 1 1 1	Yes
3	-1 0 1
0 1 -1	Yes
7	4 -4 4 -4 0 3 -3
-3 4 3 4 -4 -4 0	No
1	Yes
0	13 12 -18 15 -18 13 -17
7	
-18 13 -18 -17 12 15 13	

Note

In the first test case $\max(a_1, \dots, a_n) - \min(a_1, \dots, a_n) = 9$. Therefore, the elements can be rearranged as $[-5, -2, 3, 4]$. It is easy to see that for such an arrangement $|a_l + \dots + a_r|$ is always not greater than 7, and therefore less than 9.

In the second test case you can rearrange the elements of the array as $[-3, 2, -3, 2, 2]$. Then the maximum modulus of the sum will be reached on the subarray $[-3, 2, -3]$, and will be equal to $|-3 + 2 + -3| = |-4| = 4$, which is less than 5.

In the fourth test example, any rearrangement of the array a will be suitable as an answer, including $[-1, 0, 1]$.

D. Triangle Coloring

time limit per test: 3 seconds
memory limit per test: 512 megabytes

You are given an undirected graph consisting of n vertices and n edges, where n is divisible by 6. Each edge has a weight, which is a positive (greater than zero) integer.

The graph has the following structure: it is split into $\frac{n}{3}$ triples of vertices, the first triple consisting of vertices 1, 2, 3, the second triple consisting of vertices 4, 5, 6, and so on. Every pair of vertices from the same triple is connected by an edge. There are no edges between vertices from different triples.

You have to paint the vertices of this graph into two colors, red and blue. Each vertex should have exactly one color, there should be exactly $\frac{n}{2}$ red vertices and $\frac{n}{2}$ blue vertices. The coloring is called valid if it meets these constraints.

The weight of the coloring is the sum of weights of edges connecting two vertices with different colors.

Let W be the maximum possible weight of a valid coloring. Calculate the number of valid colorings with weight W , and print it modulo 998244353.

Input

The first line contains one integer n ($6 \leq n \leq 3 \cdot 10^5$, n is divisible by 6).

The second line contains n integers $w_1, w_2, \dots, w_n$ ($1 \leq w_i \leq 1000$) — the weights of the edges. Edge 1 connects vertices 1 and 2, edge 2 connects vertices 1 and 3, edge 3 connects vertices 2 and 3, edge 4 connects vertices 4 and 5, edge 5 connects vertices 4 and 6, edge 6 connects vertices 5 and 6, and so on.

Output

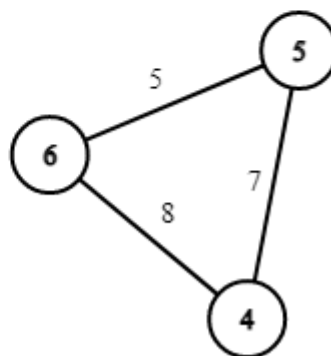
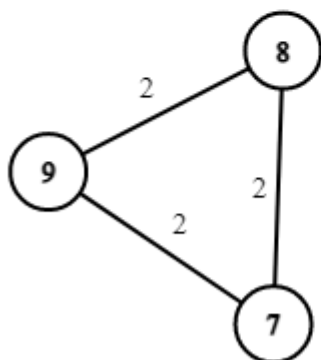
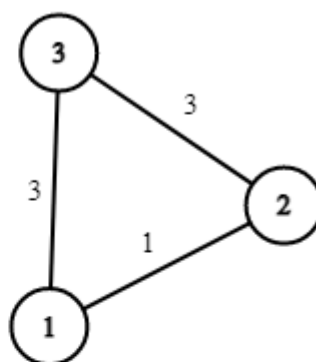
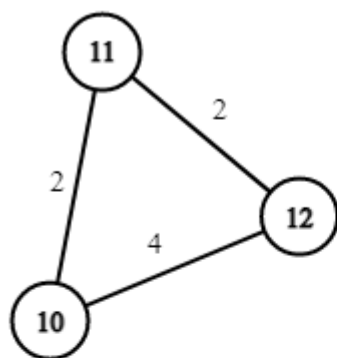
Print one integer — the number of valid colorings with maximum possible weight, taken modulo 998244353.

Examples

input	output
12 1 3 3 7 8 5 2 2 2 2 4 2	36
input	output
6 4 2 6 6 6 4	2

Note

The following picture describes the graph from the first example test.



The maximum possible weight of a valid coloring of this graph is 31.

C. Equal Frequencies

time limit per test: 2 seconds
memory limit per test: 512 megabytes

Let's call a string *balanced* if all characters that are present in it appear the same number of times. For example, "coder", "appall", and "ttttttt" are balanced, while "wowwow" and "codeforces" are not.

You are given a string s of length n consisting of lowercase English letters. Find a balanced string t of the same length n consisting of lowercase English letters that is different from the string s in as few positions as possible. In other words, the number of indices i such that $s_i \neq t_i$ should be as small as possible.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

Each test case consists of two lines. The first line contains a single integer n ($1 \leq n \leq 10^5$) — the length of the string s .

The second line contains the string s of length n consisting of lowercase English letters.

It is guaranteed that the sum of n over all test cases does not exceed 10^5 .

Output

For each test case, print the smallest number of positions where string s and a balanced string t can differ, followed by such a string t .

If there are multiple solutions, print any. It can be shown that at least one balanced string always exists.

Example

input	output
4	1
5	helno
hello	2
10	codefofced
codeforces	1
5	eeeee
eevee	0
6	appall
appall	

Note

In the first test case, the given string "hello" is not balanced: letters 'h', 'e', and 'o' appear in it once, while letter 'l' appears twice. On the other hand, string "helno" is balanced: five distinct letters are present in it, and each of them appears exactly once. Strings "hello" and "helno" differ in just one position: the fourth character. Other solutions are possible too.

In the second test case, string "codefofced" is balanced since only letters 'c', 'o', 'd', 'e', and 'f' are present in it, and each of them appears exactly twice.

In the third test case, string "eeeee" is balanced since only letter 'e' is present in it.

In the fourth test case, the given string "appall" is already balanced.

C. Flexible String

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You have a string a and a string b . Both of the strings have length n . There are **at most 10 different characters** in the string a . You also have a set Q . Initially, the set Q is empty. You can apply the following operation on the string a any number of times:

- Choose an index i ($1 \leq i \leq n$) and a lowercase English letter c . Add a_i to the set Q and then replace a_i with c .

For example, Let the string a be "abecca". We can do the following operations:

- In the first operation, if you choose $i = 3$ and $c = \mathbf{x}$, the character $a_3 = \mathbf{e}$ will be added to the set Q . So, the set Q will be $\{\mathbf{e}\}$, and the string a will be "abxcca".
- In the second operation, if you choose $i = 6$ and $c = \mathbf{s}$, the character $a_6 = \mathbf{a}$ will be added to the set Q . So, the set Q will be $\{\mathbf{e}, \mathbf{a}\}$, and the string a will be "abxccs".

You can apply any number of operations on a , but in the end, the set Q should contain **at most k different characters**. Under this constraint, you have to maximize the number of integer pairs (l, r) ($1 \leq l \leq r \leq n$) such that $a[l, r] = b[l, r]$. Here, $s[l, r]$ means the substring of string s starting at index l (inclusively) and ending at index r (inclusively).

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line contains two integers n and k ($1 \leq n \leq 10^5$, $0 \leq k \leq 10$) — the length of the two strings and the limit on different characters in the set Q .

The second line contains the string a of length n . There is **at most 10 different characters** in the string a .

The last line contains the string b of length n .

Both of the strings a and b contain only lowercase English letters. The sum of n over all test cases doesn't exceed 10^5 .

Output

For each test case, print a single integer in a line, the maximum number of pairs (l, r) satisfying the constraints.

Example

input	output
6	6
3 1	3
abc	6
abd	6
3 0	6
abc	6
abd	11
3 1	
xbb	
xcd	
4 1	
abcd	
axcb	
3 10	
abc	
abd	
10 3	
lkwhbahuqa	
qoiujoncj b	

Note

In the first case, we can select index $i = 3$ and replace it with character $c = \mathbf{d}$. All possible pairs (l, r) will be valid.

In the second case, we can't perform any operation. The 3 valid pairs (l, r) are:

1. $a[1, 1] = b[1, 1] = \mathbf{a}$,
2. $a[1, 2] = b[1, 2] = \mathbf{ab}$,
3. $a[2, 2] = b[2, 2] = \mathbf{b}$.

In the third case, we can choose index 2 and index 3 and replace them with the characters $\mathbf{c}$ and $\mathbf{d}$ respectively. The final set Q will be $\{\mathbf{b}\}$ having size 1 that satisfies the value of k . All possible pairs (l, r) will be valid.

C. Interesting Sequence

time limit per test: 1 second
memory limit per test: 256 megabytes

Petya and his friend, robot Petya++, like to solve exciting math problems.

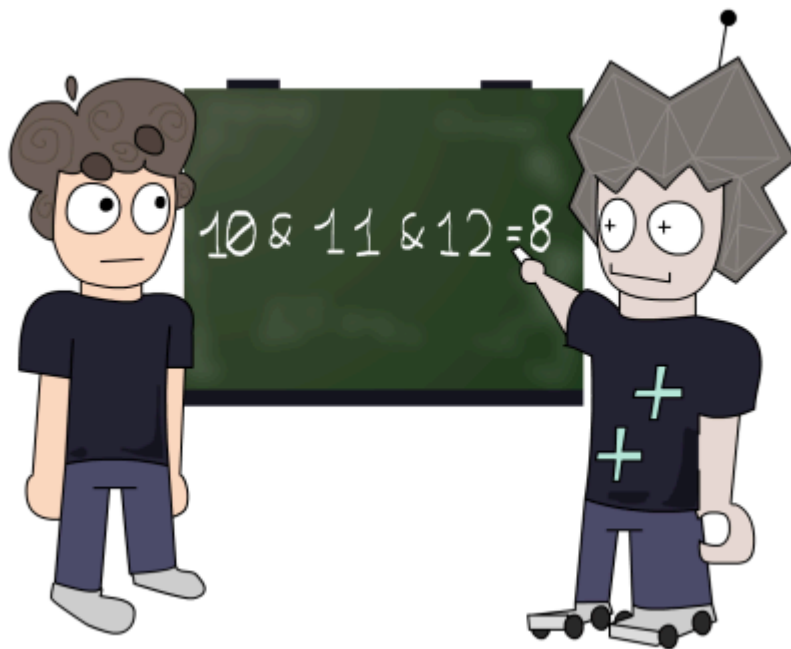
One day Petya++ came up with the numbers n and x and wrote the following equality on the board:

$$n \& (n + 1) \& \dots \& m = x,$$

where $\&$ denotes the [bitwise AND operation](#). Then he suggested his friend Petya find such a minimal m ($m \geq n$) that the equality on the board holds.

Unfortunately, Petya couldn't solve this problem in his head and decided to ask for computer help. He quickly wrote a program and found the answer.

Can you solve this difficult problem?



Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 2000$). The description of the test cases follows.

The only line of each test case contains two integers n, x ($0 \leq n, x \leq 10^{18}$).

Output

For every test case, output the smallest possible value of m such that equality holds.

If the equality does not hold for any m , print -1 instead.

We can show that if the required m exists, it does not exceed $5 \cdot 10^{18}$.

Example

input	output
5	12
10 8	10
10 10	-1
10 42	24
20 16	1152921504606846976
10000000000000000000 0	

Note

In the first example, $10 \& 11 = 10$, but $10 \& 11 \& 12 = 8$, so the answer is 12.

In the second example, $10 = 10$, so the answer is 10.

In the third example, we can see that the required m does not exist, so we have to print -1 .

E. Sending a Sequence Over the Network

time limit per test: 1 second
memory limit per test: 256 megabytes

The sequence a is sent over the network as follows:

1. sequence a is split into segments (each element of the sequence belongs to exactly one segment, each segment is a group of consecutive elements of sequence);
2. for each segment, its length is written next to it, either to the left of it or to the right of it;
3. the resulting sequence b is sent over the network.

For example, we needed to send the sequence $a = [1, 2, 3, 1, 2, 3]$. Suppose it was split into segments as follows: $[1] + [2, 3, 1] + [2, 3]$. Then we could have the following sequences:

- $b = [1, 1, 3, 2, 3, 1, 2, 3, 2]$,
- $b = [1, 1, 3, 2, 3, 1, 2, 2, 3]$,
- $b = [1, 1, 2, 3, 1, 3, 2, 2, 3]$,
- $b = [1, 1, 2, 3, 1, 3, 2, 3, 2]$.

If a different segmentation had been used, the sent sequence might have been different.

The sequence b is given. Could the sequence b be sent over the network? In other words, is there such a sequence a that converting a to send it over the network could result in a sequence b ?

Input

The first line of input data contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

Each test case consists of two lines.

The first line of the test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the size of the sequence b .

The second line of test case contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$) — the sequence b itself.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case print on a separate line:

- YES if sequence b could be sent over the network, that is, if sequence b could be obtained from some sequence a to send a over the network.
- NO otherwise.

You can output YES and NO in any case (for example, strings yEs, yes, Yes and YES will be recognized as positive response).

Example

input	output
7	YES
9	YES
1 1 2 3 1 3 2 2 3	YES
5	NO
12 1 2 7 5	YES
6	YES
5 7 8 9 10 3	NO
4	
4 8 6 2	
2	
3 1	
10	
4 6 2 1 9 4 9 3 4 2	
1	
1	

Note

In the first case, the sequence b could be obtained from the sequence $a = [1, 2, 3, 1, 2, 3]$ with the following partition: $[1] + [2, 3, 1] + [2, 3]$. The sequence b : $[1, 1, 2, 3, 1, 3, 2, 2, 3]$.

In the second case, the sequence b could be obtained from the sequence $a = [12, 7, 5]$ with the following partition: $[12] + [7, 5]$. The sequence b : $[12, 1, 2, 7, 5]$.

In the third case, the sequence b could be obtained from the sequence $a = [7, 8, 9, 10, 3]$ with the following partition: $[7, 8, 9, 10, 3]$. The sequence b : $[5, 7, 8, 9, 10, 3]$.

In the fourth case, there is no sequence a such that changing a for transmission over the network could produce a sequence b .

B. Meeting on the Line

time limit per test: 2 seconds
memory limit per test: 256 megabytes

n people live on the coordinate line, the i -th one lives at the point x_i ($1 \leq i \leq n$). They want to choose a position x_0 to meet. The i -th person will spend $|x_i - x_0|$ minutes to get to the meeting place. Also, the i -th person needs t_i minutes to get dressed, so in total he or she needs $t_i + |x_i - x_0|$ minutes.

Here $|y|$ denotes the absolute value of y .

These people ask you to find a position x_0 that minimizes the time in which all n people can gather at the meeting place.

Input

The first line contains a single integer t ($1 \leq t \leq 10^3$) — the number of test cases. Then the test cases follow.

Each test case consists of three lines.

The first line contains a single integer n ($1 \leq n \leq 10^5$) — the number of people.

The second line contains n integers $x_1, x_2, \dots, x_n$ ($0 \leq x_i \leq 10^8$) — the positions of the people.

The third line contains n integers $t_1, t_2, \dots, t_n$ ($0 \leq t_i \leq 10^8$), where t_i is the time i -th person needs to get dressed.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single real number — the optimum position x_0 . It can be shown that the optimal position x_0 is unique.

Your answer will be considered correct if its absolute or relative error does not exceed 10^{-6} . Formally, let your answer be a , the jury's answer be b . Your answer will be considered correct if

$$\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}.$$

Example

input	output
7	0
1	2
0	2.5
3	2
2	1
3 1	3
0 0	6
2	
1 4	
0 0	
3	
1 2 3	
0 0 0	
3	
1 2 3	
4 1 2	
3	
3 3 3	
5 3 3	
6	
5 4 7 2 10 4	
3 2 5 1 4 6	

Note

- In the 1-st test case there is one person, so it is efficient to choose his or her position for the meeting place. Then he or she will get to it in 3 minutes, that he or she need to get dressed.
- In the 2-nd test case there are 2 people who don't need time to get dressed. Each of them needs one minute to get to position 2.
- In the 5-th test case the 1-st person needs 4 minutes to get to position 1 (4 minutes to get dressed and 0 minutes on the way); the 2-nd person needs 2 minutes to get to position 1 (1 minute to get dressed and 1 minute on the way); the 3-rd person needs 4 minutes to get to position 1 (2 minutes to get dressed and 2 minutes on the way).

E. Split Into Two Sets

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Polycarp was recently given a set of n (number n — even) dominoes. Each domino contains two integers from 1 to n .

Can he divide all the dominoes into two sets so that all the numbers on the dominoes of each set are different? Each domino must go into exactly one of the two sets.

For example, if he has 4 dominoes: $\{1, 4\}$, $\{1, 3\}$, $\{3, 2\}$ and $\{4, 2\}$, then Polycarp will be able to divide them into two sets in the required way. The first set can include the first and third dominoes ($\{1, 4\}$ and $\{3, 2\}$), and the second set — the second and fourth ones ($\{1, 3\}$ and $\{4, 2\}$).

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The descriptions of the test cases follow.

The first line of each test case contains a single even integer n ($2 \leq n \leq 2 \cdot 10^5$) — the number of dominoes.

The next n lines contain pairs of numbers a_i and b_i ($1 \leq a_i, b_i \leq n$) describing the numbers on the i -th domino.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case print:

- YES, if it is possible to divide n dominoes into two sets so that the numbers on the dominoes of each set are different;
- NO if this is not possible.

You can print YES and NO in any case (for example, the strings yEs, yes, Yes and YES will be recognized as a positive answer).

Example

input	output
6	YES
4	NO
1 2	NO
4 3	YES
2 1	YES
3 4	NO
6	
1 2	
4 5	
1 3	
4 6	
2 3	
5 6	
2	
1 1	
2 2	
2	
1 2	
2 1	
8	
2 1	
1 2	
4 3	
4 3	
5 6	
5 7	
8 6	
7 8	
8	
1 2	
2 1	
4 3	
5 3	
5 4	
6 7	
8 6	
7 8	

Note

In the first test case, the dominoes can be divided as follows:

- First set of dominoes: $[\{1, 2\}, \{4, 3\}]$
- Second set of dominoes: $[\{2, 1\}, \{3, 4\}]$

In other words, in the first set we take dominoes with numbers 1 and 2, and in the second set we take dominoes with numbers 3 and 4.

In the second test case, there's no way to divide dominoes into 2 sets, at least one of them will contain repeated number.

D. Fixed Point Guessing

time limit per test: 1 second
memory limit per test: 256 megabytes

This is an interactive problem.

Initially, there is an array $a = [1, 2, \dots, n]$, where n is an odd positive integer. The jury has selected $\frac{n-1}{2}$ **disjoint** pairs of elements, and then the elements in those pairs are swapped. For example, if $a = [1, 2, 3, 4, 5]$, and the pairs $1 \leftrightarrow 4$ and $3 \leftrightarrow 5$ are swapped, then the resulting array is $[4, 2, 5, 1, 3]$.

As a result of these swaps, exactly one element will not change position. You need to find this element.

To do this, you can ask several queries. In each query, you can pick two integers l and r ($1 \leq l \leq r \leq n$). In return, you will be given the elements of the subarray $[a_l, a_{l+1}, \dots, a_r]$ **sorted in increasing order**.

Find the element which did not change position. You can make at most **15** queries.

The array a is fixed before the interaction and does not change after your queries.

Recall that an array b is a subarray of the array a if b can be obtained from a by deletion of several (possibly, zero or all) elements from the beginning and several (possibly, zero or all) elements from the end.

Input

Each test contains multiple test cases. The first line contains an integer t ($1 \leq t \leq 500$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains an integer n ($3 \leq n < 10^4$; n is odd) — the length of the array a .

After reading the first line of each test case, you should begin the interaction.

It is guaranteed that the sum of n over all test cases does not exceed 10^4 .

Interaction

For each test case, begin the interaction by reading the integer n .

To make a query, output `"? l r"` ($1 \leq l \leq r \leq n$) without quotes. Afterwards, you should read in $r - l + 1$ integers — the integers $a_l, a_{l+1}, \dots, a_r$, in increasing order. You can make at most 15 such queries in a single test case.

If you receive the integer `-1` instead of an answer or the integer n , it means your program has made an invalid query, has exceeded the limit of queries, or has given incorrect answer on the previous test case. Your program must terminate immediately to receive a **Wrong Answer** verdict. Otherwise you can get an arbitrary verdict because your solution will continue to read from a closed stream.

When you are ready to give the final answer, output `"! x"` ($1 \leq x \leq n$) without quotes — the element that did not change position. Giving this answer does not count towards the limit of 15 queries. Afterwards, your program must continue to solve the remaining test cases, or exit if all test cases have been solved.

After printing a query do not forget to output end of line and flush the output. Otherwise, you will get `Idleness limit exceeded`. To do this, use:

- `fflush(stdout)` or `cout.flush()` in C++;
- `System.out.flush()` in Java;
- `flush(output)` in Pascal;
- `stdout.flush()` in Python;
- see documentation for other languages.

Hacks

To make a hack, use the following format. The first line must contain an integer t ($1 \leq t \leq 500$) — the number of test cases. The description of the test cases follows.

The first line of each test case must contain an integer n ($3 \leq n < 10^4$; n is odd) — the length of the array a .

The second line of each test case must contain n space-separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the elements of a . The array a should be the result of $\frac{n-1}{2}$ disjoint swaps on the array $[1, 2, \dots, n]$.

Example

input	output
2	
5	
1 2 4 5	? 1 4
1 3 5	? 3 5
3	! 2
1	? 1 1
	! 1

Note

In the first test, the interaction proceeds as follows.

Solution	Jury	Explanation
	2	There are 2 test cases.
	5	In the first test case, the hidden array is $[4, 2, 5, 1, 3]$, with length 5.
? 1 4	1 2 4 5	The solution requests the subarray $[4, 2, 5, 1]$ in increasing order, and the jury responds with $[1, 2, 4, 5]$.
? 3 5	1 3 5	The solution requests the subarray $[5, 1, 3]$ in increasing order, and the jury responds with $[1, 3, 5]$.
! 2		The solution has somehow determined that $a_2 = 2$, and outputs it. Since the output is correct, the jury continues to the next test case.
	3	In the second test case, the hidden array is $[1, 3, 2]$, with length 3.

? 1 1	1	The solution requests the number [1] only, and the jury responds with [1].
! 1		The solution has determined that $a_1 = 1$, and outputs it. Since the output is correct and there are no more test cases, the jury and the solution exit.

Note that the line breaks in the example input and output are for the sake of clarity, and do not occur in the real interaction.

D. Maximum Product Strikes Back

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given an array a consisting of n integers. For each i ($1 \leq i \leq n$) the following inequality is true: $-2 \leq a_i \leq 2$.

You can remove any number (possibly 0) of elements from the beginning of the array and any number (possibly 0) of elements from the end of the array. You are allowed to delete the whole array.

You need to answer the question: how many elements should be removed from the beginning of the array, and how many elements should be removed from the end of the array, so that the result will be an array whose product (multiplication) of elements is **maximal**. If there is more than one way to get an array with the maximum product of elements on it, you are allowed to output **any** of them.

The product of elements of an **empty** array (array of length 0) should be assumed to be 1.

Input

The first line of input data contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases in the test.

Then the descriptions of the input test cases follow.

The first line of each test case description contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of array a .

The next line contains n integers $a_1, a_2, \dots, a_n$ ($|a_i| \leq 2$) — elements of array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output two non-negative numbers x and y ($0 \leq x + y \leq n$) — such that the product (multiplication) of the array numbers, after removing x elements from the beginning and y elements from the end, is maximal.

If there is more than one way to get the maximal product, it is allowed to output **any** of them. Consider the product of numbers on **empty** array to be 1.

Example

input	output
5	0 2
4	3 0
1 2 -1 2	2 0
3	0 1
1 1 -2	1 0
5	
2 0 -2 2 -1	
3	
-2 -1 -1	
3	
-1 -2 -2	

Note

In the first case, the maximal value of the product is 2. Thus, we can either delete the first three elements (obtain array $[2]$), or the last two and one first element (also obtain array $[2]$), or the last two elements (obtain array $[1, 2]$). Thus, in the first case, the answers fit: "3 0", or "1 2", or "0 2".

In the second case, the maximum value of the product is 1. Then we can remove all elements from the array because the value of the product on the empty array will be 1. So the answer is "3 0", but there are other possible answers.

In the third case, we can remove the first two elements of the array. Then we get the array: $[-2, 2, -1]$. The product of the elements of the resulting array is $(-2) \cdot 2 \cdot (-1) = 4$. This value is the maximum possible value that can be obtained. Thus, for this case the answer is: "2 0".

D. Make Them Equal

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You have an array of integers a of size n . Initially, all elements of the array are equal to 1. You can perform the following operation: choose two integers i ($1 \leq i \leq n$) and x ($x > 0$), and then increase the value of a_i by $\lfloor \frac{a_i}{x} \rfloor$ (i.e. make $a_i = a_i + \lfloor \frac{a_i}{x} \rfloor$).

After performing all operations, you will receive c_i coins for all such i that $a_i = b_i$.

Your task is to determine the maximum number of coins that you can receive by performing no more than k operations.

Input

The first line contains a single integer t ($1 \leq t \leq 100$) — the number of test cases.

The first line of each test case contains two integers n and k ($1 \leq n \leq 10^3$; $0 \leq k \leq 10^6$) — the size of the array and the maximum number of operations, respectively.

The second line contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^3$).

The third line contains n integers $c_1, c_2, \dots, c_n$ ($1 \leq c_i \leq 10^6$).

The sum of n over all test cases does not exceed 10^3 .

Output

For each test case, print one integer — the maximum number of coins that you can get by performing no more than k operations.

Example

input	output
4	9
4 4	0
1 7 5 2	30
2 6 5 2	167
3 0	
3 5 2	
5 4 7	
5 9	
5 2 5 6 3	
5 9 1 9 7	
6 14	
11 4 6 2 8 16	
43 45 9 41 15 38	

C. Keshi Is Throwing a Party

time limit per test: 1 second
memory limit per test: 256 megabytes

Keshi is throwing a party and he wants everybody in the party to be happy.

He has n friends. His i -th friend has i dollars.

If you invite the i -th friend to the party, he will be happy only if at most a_i people in the party are strictly richer than him and at most b_i people are strictly poorer than him.

Keshi wants to invite as many people as possible. Find the maximum number of people he can invite to the party so that every invited person would be happy.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of Keshi's friends.

The i -th of the following n lines contains two integers a_i and b_i ($0 \leq a_i, b_i < n$).

It is guaranteed that the sum of n over all test cases doesn't exceed $2 \cdot 10^5$.

Output

For each test case print the maximum number of people Keshi can invite.

Example

input	output
3	2
3	1
1 2	2
2 1	
1 1	
2	
0 0	
0 1	
2	
1 0	
0 1	

Note

In the first test case, he invites the first and the second person. If he invites all of them, the third person won't be happy because there will be more than 1 person poorer than him.

D. Say No to Palindromes

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Let's call the string **beautiful** if it does not contain a substring of length at least 2, which is a palindrome. Recall that a palindrome is a string that reads the same way from the first character to the last and from the last character to the first. For example, the strings a, bab, acca, bcabcbacb are palindromes, but the strings ab, abbbbaa, cccb are not.

Let's define **cost** of a string as the minimum number of operations so that the string becomes beautiful, if in one operation it is allowed to change any character of the string to one of the first 3 letters of the Latin alphabet (in lowercase).

You are given a string s of length n , each character of the string is one of the first 3 letters of the Latin alphabet (in lowercase).

You have to answer m queries — calculate the cost of the substring of the string s from l_i -th to r_i -th position, inclusive.

Input

The first line contains two integers n and m ($1 \leq n, m \leq 2 \cdot 10^5$) — the length of the string s and the number of queries.

The second line contains the string s , it consists of n characters, each character one of the first 3 Latin letters.

The following m lines contain two integers l_i and r_i ($1 \leq l_i \leq r_i \leq n$) — parameters of the i -th query.

Output

For each query, print a single integer — the cost of the substring of the string s from l_i -th to r_i -th position, inclusive.

Example

input	output
5 4	1
baacb	2
1 3	0
1 5	1
4 5	
2 3	

Note

Consider the queries of the example test.

- in the first query, the substring is baa, which can be changed to bac in one operation;
- in the second query, the substring is baacb, which can be changed to cbacb in two operations;
- in the third query, the substring is cb, which can be left unchanged;
- in the fourth query, the substring is aa, which can be changed to ba in one operation.

E1. Erase and Extend (Easy Version)

time limit per test: 2 seconds
memory limit per test: 256 megabytes

This is the easy version of the problem. The only difference is the constraints on n and k . You can make hacks only if all versions of the problem are solved.

You have a string s , and you can do two types of operations on it:

- Delete the last character of the string.
- Duplicate the string: $s := s + s$, where $+$ denotes concatenation.

You can use each operation any number of times (possibly none).

Your task is to find the lexicographically smallest string of length exactly k that can be obtained by doing these operations on string s .

A string a is lexicographically smaller than a string b if and only if one of the following holds:

- a is a prefix of b , but $a \neq b$;
- In the first position where a and b differ, the string a has a letter that appears earlier in the alphabet than the corresponding letter in b .

Input

The first line contains two integers n, k ($1 \leq n, k \leq 5000$) — the length of the original string s and the length of the desired string.

The second line contains the string s , consisting of n lowercase English letters.

Output

Print the lexicographically smallest string of length k that can be obtained by doing the operations on string s .

Examples

input	output
8 16 dbcadabc	dbcadabcbcadabc
input	output
4 5 abcd	aaaaa

Note

In the first test, it is optimal to make one duplication: "dbcadabc" $\rightarrow$ "dbcadabcbcadabc".

In the second test it is optimal to delete the last 3 characters, then duplicate the string 3 times, then delete the last 3 characters to make the string have length k .

"abcd" $\rightarrow$ "abc" $\rightarrow$ "ab" $\rightarrow$ "a" $\rightarrow$ "aa" $\rightarrow$ "aaaa" $\rightarrow$ "aaaaaaaa" $\rightarrow$ "aaaaaaaa" $\rightarrow$ "aaaaaa" $\rightarrow$ "aaaaaa".

A. Parsa's Humongous Tree

time limit per test: 1 second
memory limit per test: 256 megabytes

Parsa has a humongous tree on n vertices.

On each vertex v he has written two integers l_v and r_v .

To make Parsa's tree look even more majestic, Nima wants to assign a number a_v ($l_v \leq a_v \leq r_v$) to each vertex v such that the beauty of Parsa's tree is maximized.

Nima's sense of the beauty is rather bizarre. He defines the beauty of the tree as the sum of $|a_u - a_v|$ over all edges (u, v) of the tree.

Since Parsa's tree is too large, Nima can't maximize its beauty on his own. Your task is to find the **maximum** possible beauty for Parsa's tree.

Input

The first line contains an integer t ($1 \leq t \leq 250$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \leq n \leq 10^5$) — the number of vertices in Parsa's tree.

The i -th of the following n lines contains two integers l_i and r_i ($1 \leq l_i \leq r_i \leq 10^9$).

Each of the next $n - 1$ lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) meaning that there is an edge between the vertices u and v in Parsa's tree.

It is guaranteed that the given graph is a tree.

It is guaranteed that the sum of n over all test cases doesn't exceed $2 \cdot 10^5$.

Output

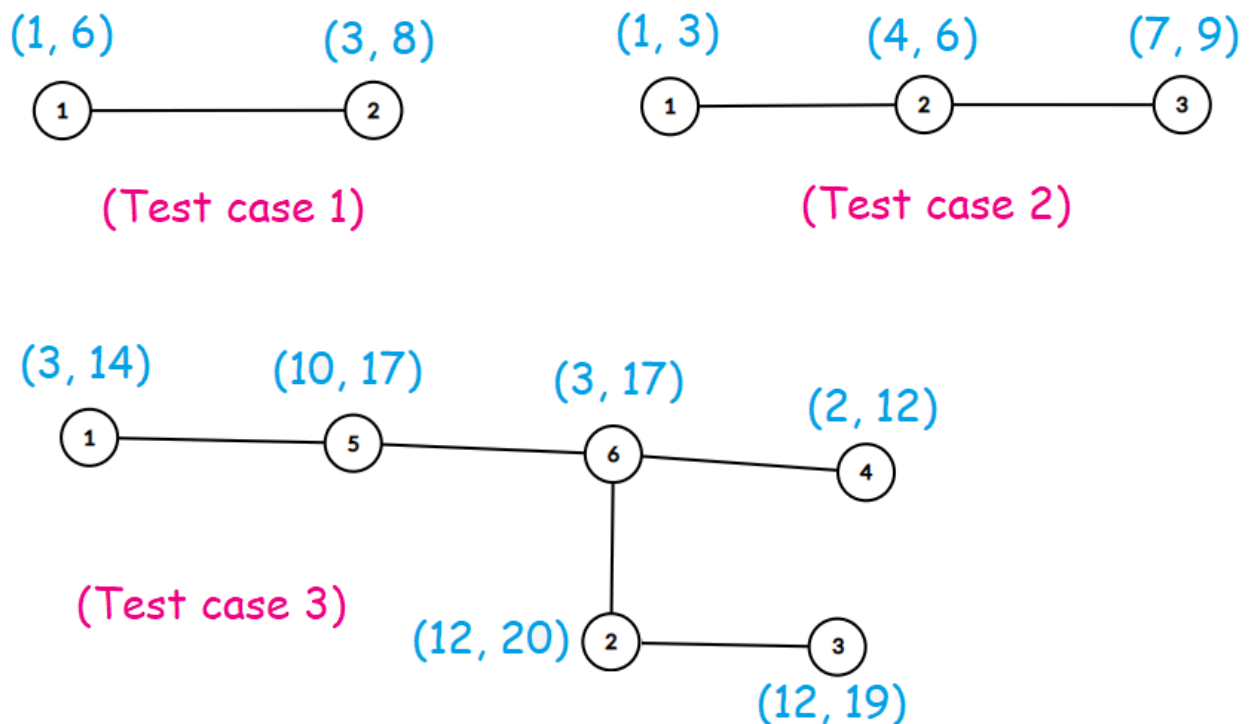
For each test case print the **maximum** possible beauty for Parsa's tree.

Example

input	output
3	7
2	8
1 6	62
3 8	
1 2	
3	
1 3	
4 6	
7 9	
1 2	
2 3	
6	
3 14	
12 20	
12 19	
2 12	
10 17	
3 17	
3 2	
6 5	
1 5	
2 6	
4 6	

Note

The trees in the example:



In the first test case, one possible assignment is $a = \{1, 8\}$ which results in $|1 - 8| = 7$.

In the second test case, one of the possible assignments is $a = \{1, 5, 9\}$ which results in a beauty of $|1 - 5| + |5 - 9| = 8$

C. Planar Reflections

time limit per test: 1 second
memory limit per test: 256 megabytes

Gaurang has grown up in a mystical universe. He is faced by n consecutive 2D planes. He shoots a particle of decay age k at the planes.

A particle can pass through a plane directly, however, every plane produces an identical copy of the particle going in the opposite direction with a decay age $k - 1$. If a particle has decay age equal to 1, it will NOT produce a copy.

For example, if there are two planes and a particle is shot with decay age 3 (towards the right), the process is as follows: (here, $D(x)$ refers to a single particle with decay age x)

1. the first plane produces a $D(2)$ to the left and lets $D(3)$ continue on to the right;
2. the second plane produces a $D(2)$ to the left and lets $D(3)$ continue on to the right;
3. the first plane lets $D(2)$ continue on to the left and produces a $D(1)$ to the right;
4. the second plane lets $D(1)$ continue on to the right ($D(1)$ cannot produce any copies).

In total, the final multiset S of particles is $\{D(3), D(2), D(2), D(1)\}$. (See notes for visual explanation of this test case.)

Gaurang is unable to cope up with the complexity of this situation when the number of planes is too large. Help Gaurang find the size of the multiset S , given n and k .

Since the size of the multiset can be very large, you have to output it modulo $10^9 + 7$.

Note: Particles can go back and forth between the planes without colliding with each other.

Input

The first line of the input contains the number of test cases t ($1 \leq t \leq 100$). Then, t lines follow, each containing two integers n and k ($1 \leq n, k \leq 1000$).

Additionally, the sum of n over all test cases will not exceed 1000, and the sum of k over all test cases will not exceed 1000. All test cases in one test are different.

Output

Output t integers. The i -th of them should be equal to the answer to the i -th test case.

Examples

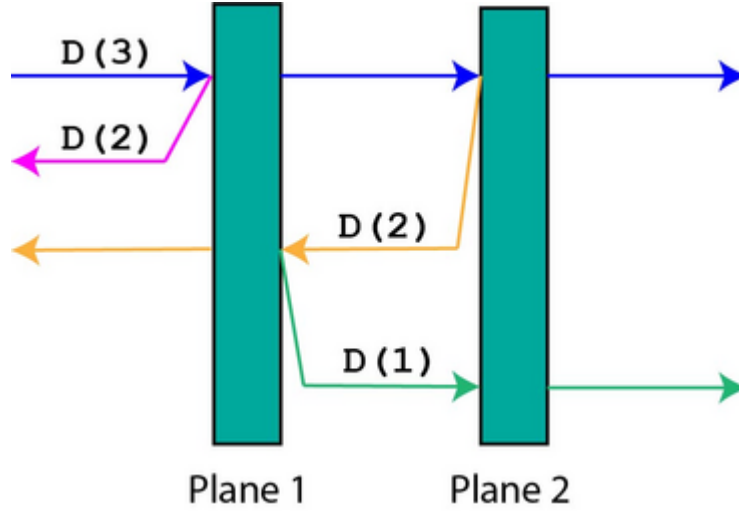
input	output
4 2 3 2 2 3 1 1 3	4 3 1 2
input	output
3 1 1 1 500 500 250	1 2 257950823

Note

Let us explain the first example with four test cases.

Test case 1: ($n = 2, k = 3$) is already explained in the problem statement.

See the below figure of this simulation. Each straight line with a different color represents the path of a different particle. As you can see, there are four distinct particles in the multiset. Note that the vertical spacing between reflected particles is for visual clarity only (as mentioned before, no two distinct particles collide with each other)



Test case 2: ($n = 2, k = 2$) is explained as follows:

1. the first plane produces a $D(1)$ to the left and lets $D(2)$ continue on to the right;
2. the second plane produces a $D(1)$ to the left and lets $D(2)$ continue on to the right;
3. the first plane lets $D(1)$ continue on to the left ($D(1)$ cannot produce any copies).

Total size of multiset obtained $\{D(1), D(1), D(2)\}$ is equal to three.

Test case 3: ($n = 3, k = 1$), there are three planes, but decay age is only one. So no new copies are produced while the one particle passes through the planes. Hence, the answer is one.

Test case 4: ($n = 1, k = 3$) there is only one plane. The particle produces a new copy to the left. The multiset $\{D(2), D(3)\}$ is of size two.

E. Advertising Agency

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Masha works in an advertising agency. In order to promote the new brand, she wants to conclude contracts with some bloggers. In total, Masha has connections of n different bloggers. Blogger numbered i has a_i followers.

Since Masha has a limited budget, she can only sign a contract with k different bloggers. Of course, Masha wants her ad to be seen by as many people as possible. Therefore, she must hire bloggers with the maximum total number of followers.

Help her, find the number of ways to select k bloggers so that the total number of their followers is maximum possible. Two ways are considered different if there is at least one blogger in the first way, which is not in the second way. Masha believes that all bloggers have different followers (that is, there is no follower who would follow two different bloggers).

For example, if $n = 4$, $k = 3$, $a = [1, 3, 1, 2]$, then Masha has two ways to select 3 bloggers with the maximum total number of followers:

- conclude contracts with bloggers with numbers 1, 2 and 4. In this case, the number of followers will be equal to $a_1 + a_2 + a_4 = 6$.
- conclude contracts with bloggers with numbers 2, 3 and 4. In this case, the number of followers will be equal to $a_2 + a_3 + a_4 = 6$.

Since the answer can be quite large, **output it modulo** $10^9 + 7$.

Input

The first line contains one integer t ($1 \leq t \leq 1000$) — the number of test cases. Then t test cases follow.

The first line of each test case contains two integers n and k ($1 \leq k \leq n \leq 1000$) — the number of bloggers and how many of them you can sign a contract with.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the number of followers of each blogger.

It is guaranteed that the sum of n over all test cases does not exceed 1000.

Output

For each test case, on a separate line output one integer — the number of ways to select k bloggers so that the total number of their followers is maximum possible.

Example

input	output
3	2
4 3	6
1 3 1 2	1
4 2	
1 1 1 1	
2 1	
1 2	

Note

The test case is explained in the statements.

In the second test case, the following ways are valid:

- conclude contracts with bloggers with numbers 1 and 2. In this case, the number of followers will be equal to $a_1 + a_2 = 2$;
- conclude contracts with bloggers with numbers 1 and 3. In this case, the number of followers will be equal to $a_1 + a_3 = 2$;
- conclude contracts with bloggers with numbers 1 and 4. In this case, the number of followers will be equal to $a_1 + a_4 = 2$;
- conclude contracts with bloggers with numbers 2 and 3. In this case, the number of followers will be equal to $a_2 + a_3 = 2$;
- conclude contracts with bloggers with numbers 2 and 4. In this case, the number of followers will be equal to $a_2 + a_4 = 2$;
- conclude contracts with bloggers with numbers 3 and 4. In this case, the number of followers will be equal to $a_3 + a_4 = 2$.

In the third test case, the following ways are valid:

- concludes a contract with a blogger with the number 2. In this case, the number of followers will be equal to $a_2 = 2$.

A. Row GCD

time limit per test: 2 seconds
memory limit per test: 512 megabytes

You are given two positive integer sequences $a_1, \dots, a_n$ and $b_1, \dots, b_m$. For each $j = 1, \dots, m$ find the greatest common divisor of $a_1 + b_j, \dots, a_n + b_j$.

Input

The first line contains two integers n and m ($1 \leq n, m \leq 2 \cdot 10^5$).

The second line contains n integers $a_1, \dots, a_n$ ($1 \leq a_i \leq 10^{18}$).

The third line contains m integers $b_1, \dots, b_m$ ($1 \leq b_j \leq 10^{18}$).

Output

Print m integers. The j -th of them should be equal to $\text{GCD}(a_1 + b_j, \dots, a_n + b_j)$.

Example

input	output
4 4 1 25 121 169 1 2 7 23	2 3 8 24

C. Chocolate Bunny

time limit per test: 1 second
memory limit per test: 256 megabytes

This is an interactive problem.

We hid from you a permutation p of length n , consisting of the elements from 1 to n . You want to guess it. To do that, you can give us 2 different indices i and j , and we will reply with $p_i \bmod p_j$ (remainder of division p_i by p_j).

We have enough patience to answer at most $2 \cdot n$ queries, so you should fit in this constraint. Can you do it?

As a reminder, a permutation of length n is an array consisting of n distinct integers from 1 to n in arbitrary order. For example, $[2, 3, 1, 5, 4]$ is a permutation, but $[1, 2, 2]$ is not a permutation (2 appears twice in the array) and $[1, 3, 4]$ is also not a permutation ($n = 3$ but there is 4 in the array).

Input

The only line of the input contains a single integer n ($1 \leq n \leq 10^4$) — length of the permutation.

Interaction

The interaction starts with reading n .

Then you are allowed to make at most $2 \cdot n$ queries in the following way:

- `"? x y"` ($1 \leq x, y \leq n, x \neq y$).

After each one, you should read an integer k , that equals $p_x \bmod p_y$.

When you have guessed the permutation, print a single line `"! "` (without quotes), followed by array p and quit.

After printing a query do not forget to output end of line and flush the output. Otherwise, you will get `Idleness limit exceeded`. To do this, use:

- `fflush(stdout)` or `cout.flush()` in C++;
- `System.out.flush()` in Java;
- `flush(output)` in Pascal;
- `stdout.flush()` in Python;
- see documentation for other languages.

Exit immediately after receiving `"-1"` and you will see `Wrong answer` verdict. Otherwise you can get an arbitrary verdict because your solution will continue to read from a closed stream.

Hack format

In the first line output n ($1 \leq n \leq 10^4$). In the second line print the permutation of n integers $p_1, p_2, \dots, p_n$.

Example

input	output
3	? 1 2
1	? 3 2
2	? 1 3
1	? 2 1
0	! 1 3 2

C. Good Subarrays

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$ consisting of integers from 0 to 9. A subarray $a_l, a_{l+1}, a_{l+2}, \dots, a_{r-1}, a_r$ is good if the sum of elements of this subarray is equal to the length of this subarray ($\sum_{i=l}^r a_i = r - l + 1$).

For example, if $a = [1, 2, 0]$, then there are 3 good subarrays: $a_{1..1} = [1]$, $a_{2..3} = [2, 0]$ and $a_{1..3} = [1, 2, 0]$.

Calculate the number of good subarrays of the array a .

Input

The first line contains one integer t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains one integer n ($1 \leq n \leq 10^5$) — the length of the array a .

The second line of each test case contains a string consisting of n decimal digits, where the i -th digit is equal to the value of a_i .

It is guaranteed that the sum of n over all test cases does not exceed 10^5 .

Output

For each test case print one integer — the number of good subarrays of the array a .

Example

input	output
3	3
3	6
120	1
5	
11011	
6	
600005	

Note

The first test case is considered in the statement.

In the second test case, there are 6 good subarrays: $a_{1..1}$, $a_{2..2}$, $a_{1..2}$, $a_{4..4}$, $a_{5..5}$ and $a_{4..5}$.

In the third test case there is only one good subarray: $a_{2..6}$.

B. Array Walk

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$, consisting of n **positive** integers.

Initially you are standing at index 1 and have a score equal to a_1 . You can perform two kinds of moves:

1. move right — go from your current index x to $x + 1$ and add a_{x+1} to your score. This move can only be performed if $x < n$.
2. move left — go from your current index x to $x - 1$ and add a_{x-1} to your score. This move can only be performed if $x > 1$. **Also, you can't perform two or more moves to the left in a row.**

You want to perform **exactly** k moves. Also, there should be no more than z moves to the left among them.

What is the maximum score you can achieve?

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of testcases.

The first line of each testcase contains three integers n, k and z ($2 \leq n \leq 10^5$, $1 \leq k \leq n - 1$, $0 \leq z \leq \min(5, k)$) — the number of elements in the array, the total number of moves you should perform and the maximum number of moves to the left you can perform.

The second line of each testcase contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^4$) — the given array.

The sum of n over all testcases does not exceed $3 \cdot 10^5$.

Output

Print t integers — for each testcase output the maximum score you can achieve if you make exactly k moves in total, no more than z of them are to the left and there are no two or more moves to the left in a row.

Example

input	output
4	15
5 4 0	19
1 5 4 3 2	150
5 4 1	56
1 5 4 3 2	
5 4 4	
10 20 30 40 50	
10 7 3	
4 6 8 2 9 9 7 4 10 9	

Note

In the first testcase you are not allowed to move left at all. So you make four moves to the right and obtain the score $a_1 + a_2 + a_3 + a_4 + a_5$.

In the second example you can move one time to the left. So we can follow these moves: right, right, left, right. The score will be $a_1 + a_2 + a_3 + a_2 + a_3$.

In the third example you can move four times to the left but it's not optimal anyway, you can just move four times to the right and obtain the score $a_1 + a_2 + a_3 + a_4 + a_5$.

A. Orac and LCM

time limit per test: 3 seconds
memory limit per test: 256 megabytes

For the multiset of positive integers $s = \{s_1, s_2, \dots, s_k\}$, define the Greatest Common Divisor (GCD) and Least Common Multiple (LCM) of s as follow:

- $\gcd(s)$ is the maximum positive integer x , such that all integers in s are divisible on x .
- $\text{lcm}(s)$ is the minimum positive integer x , that divisible on all integers from s .

For example, $\gcd(\{8, 12\}) = 4$, $\gcd(\{12, 18, 6\}) = 6$ and $\text{lcm}(\{4, 6\}) = 12$. Note that for any positive integer x , $\gcd(\{x\}) = \text{lcm}(\{x\}) = x$.

Orac has a sequence a with length n . He come up with the multiset $t = \{\text{lcm}(\{a_i, a_j\}) \mid i < j\}$, and asked you to find the value of $\gcd(t)$ for him. In other words, you need to calculate the GCD of LCMs of all pairs of elements in the given sequence.

Input

The first line contains one integer n ($2 \leq n \leq 100\,000$).

The second line contains n integers, $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 200\,000$).

Output

Print one integer: $\gcd(\{\text{lcm}(\{a_i, a_j\}) \mid i < j\})$.

Examples

input 2 1 1	output 1
input 4 10 24 40 80	output 40
input 10 540 648 810 648 720 540 594 864 972 648	output 54

Note

For the first example, $t = \{\text{lcm}(\{1, 1\})\} = \{1\}$, so $\gcd(t) = 1$.

For the second example, $t = \{120, 40, 80, 120, 240, 80\}$, and it's not hard to see that $\gcd(t) = 40$.

C. Linova and Kingdom

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Writing light novels is the most important thing in Linova's life. Last night, Linova dreamed about a fantastic kingdom. She began to write a light novel for the kingdom as soon as she woke up, and of course, she is the queen of it.



There are n cities and $n - 1$ two-way roads connecting pairs of cities in the kingdom. From any city, you can reach any other city by walking through some roads. The cities are numbered from 1 to n , and the city 1 is the capital of the kingdom. So, the kingdom has a tree structure.

As the queen, Linova plans to choose **exactly** k cities developing industry, while the other cities will develop tourism. The capital also can be either industrial or tourism city.

A meeting is held in the capital once a year. To attend the meeting, each **industry city** sends an envoy. All envoys will follow the shortest path from the departure city to the capital (which is unique).

Traveling in tourism cities is pleasant. For each envoy, his *happiness* is equal to the number of **tourism cities** on his path.

In order to be a queen loved by people, Linova wants to choose k cities which can maximize the sum of *happineses* of all envoys. Can you calculate the maximum sum for her?

Input

The first line contains two integers n and k ($2 \leq n \leq 2 \cdot 10^5$, $1 \leq k < n$) — the number of cities and industry cities respectively.

Each of the next $n - 1$ lines contains two integers u and v ($1 \leq u, v \leq n$), denoting there is a road connecting city u and city v .

It is guaranteed that from any city, you can reach any other city by the roads.

Output

Print the only line containing a single integer — the maximum possible sum of *happineses* of all envoys.

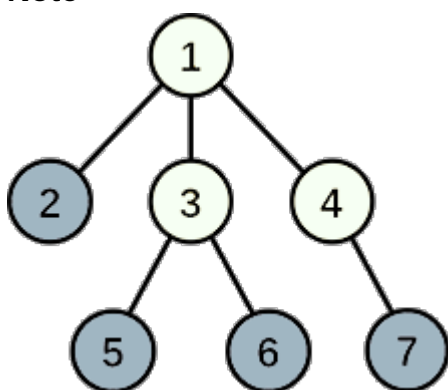
Examples

input	output
7 4 1 2 1 3 1 4 3 5 3 6 4 7	7

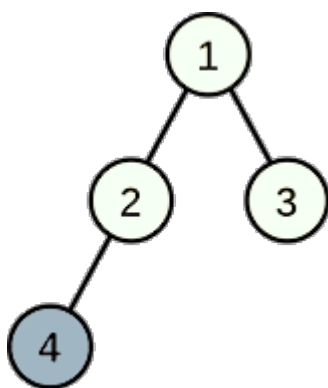
input	output
4 1 1 2 1 3 2 4	2

input	output
8 5 7 5 1 7 6 1 3 7 8 3 2 1 4 5	9

Note



In the first example, Linova can choose cities 2, 5, 6, 7 to develop industry, then the *happiness* of the envoy from city 2 is 1, the *happiness* of envoys from cities 5, 6, 7 is 2. The sum of *happinesses* is 7, and it can be proved to be the maximum one.



In the second example, choosing cities 3, 4 developing industry can reach a sum of 3, but remember that Linova plans to choose **exactly** k cities developing industry, then the maximum sum is 2.

C. Kuroni and Impossible Calculation

time limit per test: 1 second
memory limit per test: 256 megabytes

To become the king of Codeforces, Kuroni has to solve the following problem.

He is given n numbers $a_1, a_2, \dots, a_n$. Help Kuroni to calculate $\prod_{1 \leq i < j \leq n} |a_i - a_j|$. As result can be very big, output it modulo m .

If you are not familiar with short notation, $\prod_{1 \leq i < j \leq n} |a_i - a_j|$ is equal to $|a_1 - a_2| \cdot |a_1 - a_3| \cdot \dots \cdot |a_1 - a_n| \cdot |a_2 - a_3| \cdot |a_2 - a_4| \cdot \dots \cdot |a_2 - a_n| \cdot \dots \cdot |a_{n-1} - a_n|$. In other words, this is the product of $|a_i - a_j|$ for all $1 \leq i < j \leq n$.

Input

The first line contains two integers n, m ($2 \leq n \leq 2 \cdot 10^5, 1 \leq m \leq 1000$) — number of numbers and modulo.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$).

Output

Output the single number — $\prod_{1 \leq i < j \leq n} |a_i - a_j| \bmod m$.

Examples

input 2 10 8 5	output 3
input 3 12 1 4 5	output 0
input 3 7 1 4 9	output 1

Note

In the first sample, $|8 - 5| = 3 \equiv 3 \bmod 10$.

In the second sample, $|1 - 4| \cdot |1 - 5| \cdot |4 - 5| = 3 \cdot 4 \cdot 1 = 12 \equiv 0 \bmod 12$.

In the third sample, $|1 - 4| \cdot |1 - 9| \cdot |4 - 9| = 3 \cdot 8 \cdot 5 = 120 \equiv 1 \bmod 7$.